
Regression Analysis
With
R And Easystats

AUTHOR

MAULIK BHATT

*S P JAIN INSTITUTE OF MANAGEMENT & RESEARCH
MUMBAI*

2024
MAULIK BHATT

Copyright © 2024 Maulik Bhatt

Contents

List of Figures	v
List of Tables	vii
Preface	1
Acknowledgement	3
1. Introduction	5
1.1. Importance of R for Econometrics and Statistics	6
2. Introduction to Econometrics and Regression Analysis	9
2.1. Introduction to R	10
2.2. The Role of Econometrics in Economics	11
2.2.1. Econometric Methodology	12
2.2.2. Applications of Econometrics	13
3. Getting Started with R for Econometrics	15
3.1. A Brief History of R	16
3.2. How to Install R	16
3.3. How to Install RStudio	17
3.4. Useful Settings in RStudio	17
3.5. The Concept of Working Directory	18
3.6. Quarto	18
3.7. Optional: Installing $\text{\LaTeX}$	19
3.8. Your First Interaction with R	20
3.9. Installing Packages	21
3.10. Loading and Manipulating Data	21
3.10.1. Manipulating Data	22
3.10.2. Selecting Columns from Dataset	23
3.10.3. Selecting Rows from a Dataset (aka filtering)	24
3.10.4. Arrange Data	25
3.10.5. Create a New Variable	26
3.11. Summarize a Data	27
4. Simple Linear Regression	29
4.1. Introduction	30
4.2. Unveiling Relationships: The Purpose of Linear Regression	30
4.3. Unveiling the Assumptions Behind Linear Regression	32
4.4. Example in R	33
4.4.1. Interpretation of Regression Model	36

4.4.2. Non-Linear Relationship in Regression	36
4.5. Interval Estimation and Hypothesis Testing	37
4.6. Hypothesis Testing	39
4.7. Prediction	41
5. Multiple Linear Regression	47
5.0.1. Introduction	48
5.1. Assumptions of Multiple Linear Regression	48
5.2. Multiple Linear Regression: A Practical Example	49
6. Regression Analysis with Dummy Variables	55
6.1. Dummy Variables	56
6.2. Dummy Variables in Regression	56
6.3. Recoding Dummy Variables	59
6.4. Dummy Variable Trap	60
7. Heteroscedasticity and Robust Regression	65
7.1. Formal Test for Heteroscedasticity	69
8. Time Series Regression	71
9. Introduction to Logistic Regression	73
10. Model Evaluation and Selection	75
11. Practical Tips and Resources for Econometric Regression	77
11.1. Data Cleaning	78
11.2. Finding Missing Values	80
11.3. Date Stored as Numbers	81
11.4. Identifying Outliers in Data	82
12. Conclusion	83
 Appendices	 85
A. R packages for econometric regression analysis and additional resources	85
B. Data sets used in the book's examples	89
B.0.1. Introduction	89
C. How to Build This Book Locally	91
C.0.1. Introduction	91
References	95

List of Figures

3.1. How to Disable Saving Workspace	17
3.2. RStudio Panes	20
3.3. Types of RStudio Projects	21
4.1. Scatterplot of X and Y	34
C.1. Download the book from Github	91
C.2. Build book using tinytex	92

List of Tables

3.2. Selecting Columns from am to carb from mtcars dataset	23
3.3. Selecting Rows from mtcars dataset where value of cyl is 6	24
3.4. Arrange the Data in Ascending Order of mpg	25
3.5. Arrange the Data in Descending Order of mpg	25
3.6. Create a New Variable in mtcars dataset	26
3.7. Average Miles Per Gallon of Cars in USA	27
3.8. Miles Per Gallon for Different Values of Cylinder	27
4.2. Regression Parameters	35
4.3. Regression Effectiveness	36
4.3. Regression Effectiveness	36
4.4. Confidence Interval of Model Coefficients	39
5.2. Multiple Regression Parameters	52
11.2. Making Unique Names in a Dataset	79
11.3. Making Clean Names in a Dataset using janitor package	79
11.4. Summary of ‘Species’ column of ‘iris’ dataset	81

Preface

Acknowledgement

I would like to express my heartfelt gratitude to the individuals who have played a significant role in the creation of this book on Regression Analysis with R. Their guidance, knowledge, and support have been invaluable throughout this journey.

First and foremost, I extend my deepest appreciation to my esteemed professors, [Professor's Name], [Professor's Name], and [Professor's Name]. Their expertise in statistics and econometrics has shaped my understanding of quantitative analysis and provided a solid foundation for this book. Their dedication to teaching and their unwavering commitment to fostering academic excellence have been instrumental in my growth as a student of econometrics.

I would like to extend special thanks to my colleague and dear friend, [Colleague's Name]. Their introduction to the world of R programming language opened up a world of possibilities for me. Their patience, willingness to share their knowledge, and countless hours spent assisting me with R-related challenges have been indispensable. I am grateful for their unwavering support and the collaborative environment we fostered, which significantly enriched my learning experience.

A heartfelt appreciation goes to the developers of R, an open-source programming language that has revolutionized the field of data analysis and statistical computing. Their tireless efforts in creating

and continuously improving R have made it an indispensable tool for researchers and analysts worldwide. Without their dedication, this book would not have been possible.

I would also like to express my gratitude to the developers of the R packages that have been instrumental in the analysis and visualization techniques presented in this book. Their commitment to excellence, innovation, and user-friendly implementations has immensely contributed to the field of regression analysis. Their packages have not only expanded the capabilities of R but have also facilitated the seamless integration of econometric methodologies into practical applications.

Finally, I extend my heartfelt thanks to my family and friends who have supported me throughout this writing process. Their encouragement, understanding, and belief in my abilities have been a constant source of motivation.

To all those mentioned above, and to anyone else who has contributed to this book in any way, I offer my deepest appreciation. Your guidance, knowledge, and support have been invaluable in shaping this work and have helped me fulfill my goal of sharing the beauty and importance of regression analysis with R.

Thank you.

[Your Name]

1

Introduction

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Importance of regression analysis in econometrics*
 - *Overview of the book's structure and goals*
 - *Introduction to R programming language and its relevance to econometric analysis*
-

Econometrics is a combination of two words: Econo + Metric. Econo refers to concepts of economics, while metric refers to measurement. Let's take an example of the law of demand from microeconomic theory. We know the demand of a commodity will decrease if the price of the commodity increases. But we don't know how much the demand will decrease, given the increase in the price is one unit

(i.e., one Dollar, or one Euro, etc.). In Econometrics, we measure such increase or decrease using various experiments. In these experiments, we observe how much the demand for a commodity increase or decrease in response to a unit change (increase or decrease) in price of the commodity.

1.1 Importance of R for Econometrics and Statistics

Many software are available for statistical and econometric analysis. Some of the popular software include SPSS, SAS, Stata, MATLAB, etc. These are all proprietary software. Free and open source alternatives include python and R, which are programming languages. The users need to type the commands in order to perform various tasks/analysis in these programming languages.

Python is a general purpose programming language, while R is a statistical programming language. R also has many packages (i.e., add-ons, which enhance the functionality of R). As of writing this, there are 19697 packages available on CRAN (the Comprehensive R Archive Network, which is the central authority to decide about R programming language). Even more packages are available on [github](#), which is an Internet hosting service for software development and version control using the version control system Git. Because of such extensive support, I have chosen R to conduct econometric analysis for this book.

In particular, I will use **easystats** set of packages, which are designed for various statistical analysis tasks. We will see how **easystats** make our tasks easier in R.

I will also use **quarto**, which is a document preparation system, based on pandoc's markdown. The benefit of **quarto** is that we can keep our analysis and prose in the same document. At the end, when we click on the **build** button, it renders the entire book in either pdf or HTML format. It is also possible to render the book in Microsoft Word format, but this output format has got limited

support. It is more convenient to design and customize the output in HTML and PDF formats. We will discuss more about this in the chapter of Getting Started with R.

2

Introduction to Econometrics and Regression Analysis

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Understanding the basics of econometrics*
- *Role of regression analysis in econometric modeling*
- *Overview of the regression analysis process*
- *Introduction to R for econometric analysis*

Basics of Econometrics

Role of Regression

2.1 Introduction to R

Imagine economics as a world of ideas. Economists come up with theories that explain how the economy works, like how changes in price affect demand. These theories are great for understanding the general principles, but they often lack specifics. They can't tell you exactly how much demand will drop if the price goes up by a certain amount.

That's where econometrics comes in. It acts as a bridge between these economic theories and the messy, real world. The word "econometrics" is made of two words: Economics, and Metrics. Metrics means measurement. Thus, econometrics means introducing measurement inside economics. Econometrics uses tools from statistics and math to turn those economic theories into something measurable. Here's the breakdown:

- **Quantify relationships:** Econometrics doesn't just say "demand goes down with price increase," it can tell you by how much. It uses statistical methods like regression analysis to find the exact mathematical relationship between price and demand based on real-world data.
- **Test theories:** Economic theories are like educated guesses. Econometrics allows us to test those guesses with real data. Using statistical tests, econometricians can see if the data actually supports the theory or not. Maybe higher prices don't always decrease demand as much as the theory predicted.

- **Informed decisions:** By giving us numbers and concrete evidence, econometrics helps us make better decisions about the economy. Policymakers can use econometric models to see how changes in interest rates might affect inflation, or businesses can use them to predict how much demand there will be for their product if they raise the price.

This chapter is like a map to understanding econometrics. It will introduce you to the core methods (like regression analysis) and show you how they're used in different areas like business forecasting or analyzing the labor market.

In short, econometrics takes economic theories from the world of ideas and translates them into tools we can use to understand and navigate the real economic world.

2.2 The Role of Econometrics in Economics

Economic theories often propose qualitative relationships between variables. For instance, a theory might suggest that an increase in income leads to a rise in consumption. Econometrics allows us to quantify this relationship by estimating the magnitude of change in consumption for a given change in income. This is achieved by constructing mathematical models that represent the theoretical relationships and then employing statistical techniques to estimate the unknown parameters within these models.

Economic Theories as Qualitative Descriptions: Economic theories provide valuable insights into how the economy functions. However, these theories often describe relationships between variables in qualitative terms. For example, a theory might simply state that there's a positive relationship between income and consumption, but it doesn't tell us by how much consumption increases for a specific increase in income.

Econometrics: Bridging the Gap with Quantification: This is where econometrics steps in. It acts as a bridge between these qualitative theories and the quantitative world. Econometricians achieve this by:

- **Mathematical Modeling:** They construct mathematical models that represent the theoretical relationships between variables. These models typically involve equations that express the dependent variable (e.g., consumption) as a function of the independent variable (e.g., income) and potentially other relevant factors.
- **Statistical Estimation:** The models contain unknown parameters that need to be estimated using real-world data. Econometricians employ various statistical techniques, like regression analysis, to estimate these parameters. This allows them to quantify the magnitude and direction of the relationship between the variables.

In essence, econometrics takes the qualitative predictions of economic theories and transforms them into measurable and testable relationships. By estimating the unknown parameters within the models, econometricians can tell us not just that consumption goes up with income, but by exactly how much it's expected to rise for a given increase in income. This quantitative information proves invaluable for various applications, such as economic forecasting, policy analysis, and business decision-making.

2.2.1. Econometric Methodology

The econometric toolbox is vast, but some fundamental methods include:

- **Regression Analysis:** This technique explores the relationship between a dependent variable (the variable being explained) and one or more independent variables (explanatory variables).

By estimating the coefficients of the independent variables, we can quantify their impact on the dependent variable.

- **Hypothesis Testing:** Econometrics allows us to statistically test economic hypotheses. This involves formulating a null hypothesis (typically, no relationship exists) and an alternative hypothesis (a specific relationship exists), and then using statistical tests to determine if the data supports rejecting the null hypothesis in favor of the alternative.
- **Time Series Analysis:** This method deals with analyzing data collected over time, allowing us to understand trends, seasonality, and potential causal relationships between economic variables.

2.2.2. Applications of Econometrics

Econometrics has a wide range of applications in various economic fields. Here are some prominent examples:

- **Demand Forecasting:** Businesses can utilize econometric models to forecast consumer demand for their products based on factors like income, price, and advertising expenditures.
- **Monetary Policy Analysis:** Central banks employ econometric models to assess the impact of interest rate changes on inflation, unemployment, and economic growth.
- **Labor Market Analysis:** Econometricians use these methods to examine the effects of minimum wage laws, training programs, and other policies on employment levels and wages.

3

Getting Started with R for Econometrics

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Introduction to R programming language and its ecosystem*
 - *Setting up the R environment and installing necessary packages*
 - *Loading and manipulating data in R for econometric analysis*
 - *Exploring data visualization techniques using R*
-

3.1 A Brief History of R

John Chambers, working at Bell Laboratories, developed S programming language for statistical analysis. This programming language was incorporated in the commercial program S-plus. Inspired by this programming language, two statistics professors Robert Gentleman and Ross Ihaka developed a reduced version of S, which they named R.

R was first released in 1993. In 1995, another statistician Martin Mächler convinced Gentleman and Ihaka to make R source code available to general public. As a result, R became free and open source under GNU General Public License.

R also has a robust community of users. As of writing this, there are 20607 user-developed packages on Comprehensive R Archive Network (CRAN), which distributes the R source code and packages. Apart from CRAN, many more packages are also available on code sharing platform github. Users can download packages from github as well. The Appendix A explains how to install packages from CRAN and from github.

3.2 How to Install R

You can install R from the website of [CRAN](https://cran.r-project.org/). Downloadable files and instructions to download and install R for all the major operating systems are available on this website. R can be downloaded and installed on Windows, Mac and Linux platforms.

It is possible to use R right from within this R Graphical User Interface (R-Gui). But such a work will not be reproducible. It means, you will be able to work on your system, but you won't be able to share that work with others. IDE like RStudio makes it much easier to work in R, and to share your work with others.

3.3 How to Install RStudio

Once you have installed R for your operating system, you can visit the website of [RStudio](https://www.rstudio.com/). Just like R, RStudio is also available on Windows, Mac and Linux platforms.

3.4 Useful Settings in RStudio

I would recommend you not to save the workspace in RStudio. This setting can be disabled in RStudio as follows:

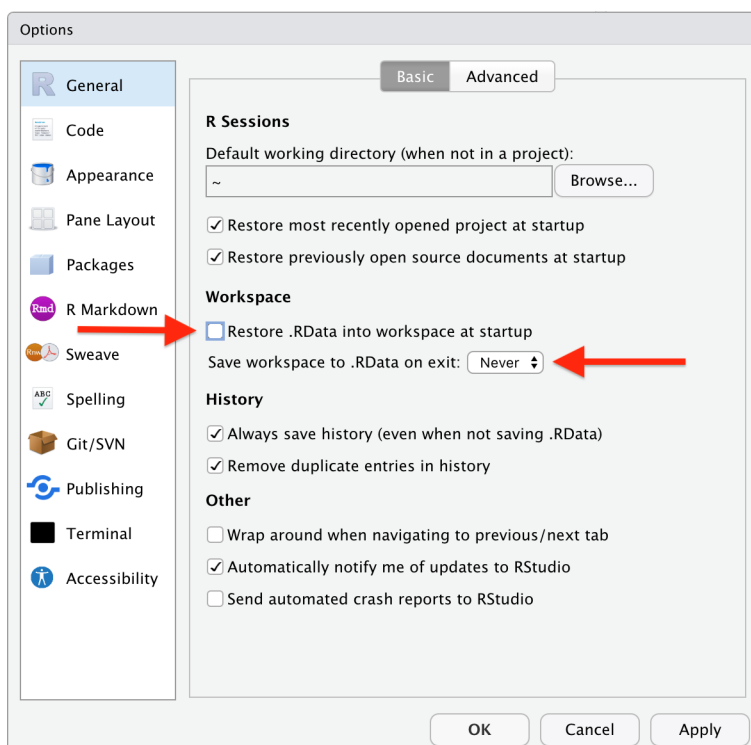


Figure 3.1.: How to Disable Saving Workspace

This setting (of not saving your workspace) seems counter-intuitive. But it helps you in the longer term. Assume you have to share your work with someone else. The other person might not have the same settings of computer in his/her system. So, the work on your system might look different

in that system. Instead, you should save the R Script (or even better, a Quarto document) in your system, and should share it.

3.5 The Concept of Working Directory

R has a concept of working directory. When you save your R script and your work, it gets saved in the “working directory” of R. You can find and change the working directory using R commands.

```
# Find the working directory

getwd()

#Change the working directory

#setwd("Path/to/another/folder")
```

3.6 Quarto

Quarto is a new open-source scientific and technical publishing system developed by Posit (the maker of RStudio). Quarto is designed to be useful to anyone who wants to create reproducible documents. A Quarto document contains both - the code and the prose. For example, if you are running regression analysis in R, R Script will only contain your code, but a Quarto document will contain your R code and your interpretation of the regression results. Quarto runs computations into separate pluggable language “engines”, which helps make this cross-language functionality easier to support .

Here are some points that emphasize the reproducibility of Quarto over R scripts:

1. **Cross-language support:** Quarto is designed to work with multiple languages, including Python, bash, Julia, C, SQL, and more. This makes it easier to work with different languages

in the same document.

2. **Built-in output formats:** Quarto generates the output in various formats like Microsoft Word, HTML, PDF, beamer, revealjs, etc. It also has many options for customizing each format.
3. **Native features for special project types:** Quarto has native features for special project types like websites, books, and blogs. This means that you don't have to rely on external packages. As a matter of fact, this book is written entirely using Quarto in RStudio.
4. **Easier rendering:** Quarto isn't an R package. It's a command-line interface that makes it much easier to work with Quarto documents outside of the RStudio IDE. You can also use Quarto in other IDEs like VS Code.

These features make Quarto a better choice than R scripts when it comes to reproducibility. With Quarto, you can easily create documents that are easy to reproduce and share with others.

3.7 Optional: Installing $\text{\LaTeX}$

If you want to work in Quarto, and want to generate PDF output, $\text{\LaTeX}$ is required. There are two popular $\text{\LaTeX}$ distributions: MiKTeX and TeX Live. However, I prefer another $\text{\LaTeX}$ distribution, called TinyTeX. It's a light weight distribution of $\text{\LaTeX}$, and works well with R. You can install it using R as follows:

```
#install tinytex R package  
  
install.packages("tinytex")  
  
#load it in R  
  
library(tinytex)
```

```
#use this package to install LaTeX compiler TinyTeX

install_tinytex()
```

These commands install the most commonly used $\text{\LaTeX}$ packages into your system. However, if you want to install all the $\text{\LaTeX}$ packages, you can do so by using `install_tinytex(bundle = "TinyTeX-2")` instead of `install_tinytex()` function above.

3.8 Your First Interaction with R

After installing R and RStudio, you would start RStudio (not R). RStudio has four parts in the screen. The four parts (called “panes”) are source pane, console pane, environment pane and files pane. You can customize each pane using **Tools Menu > Global Options > Pane Layout**. The RStudio screen looks as follows:

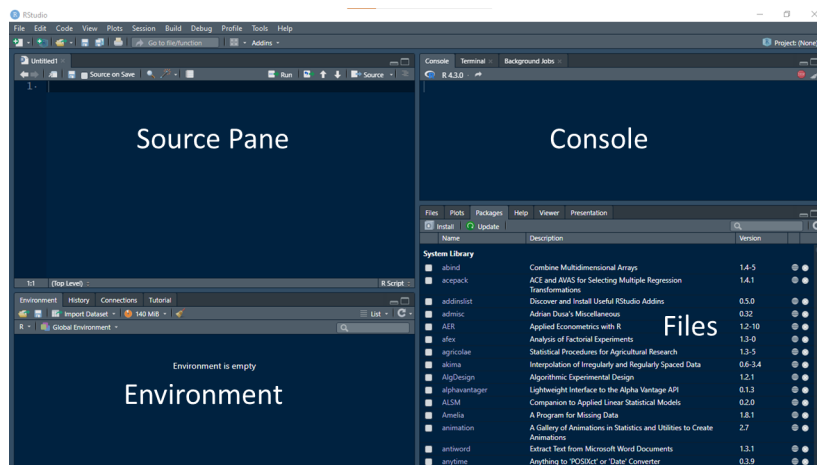


Figure 3.2.: RStudio Panes

When you open a new file in RStudio using the **Files Menu > New File**, you are presented with many options like R Script, R Markdown Document, Quarto Document, etc. I would suggest you to choose R Markdown Document or Quarto Document, because in these two file formats, you can write both - your analysis and your prose. Your document will be saved in your working directory.

In RStudio's File Menu, you will also find another option: **New Project**. This is a considerable improvement over a new file - be it an R Script or a Quarto Document. This is because it changes the working directory of the project to the folder, where `.Rproj` file of the project is stored. You can share this folder with anyone, and the code will work just fine on that computer also.

When you go to **File Menu > New Project > New Directory**, you get options as follows:

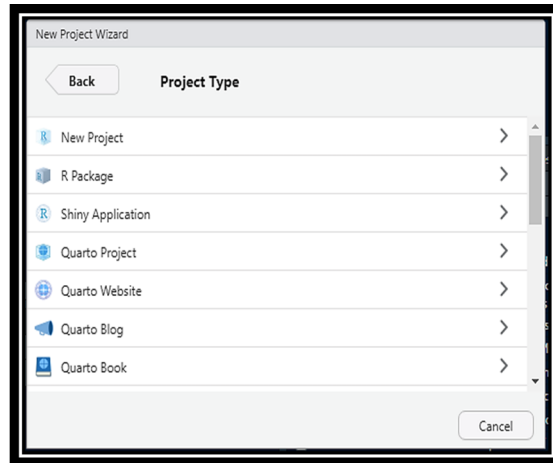


Figure 3.3.: Types of RStudio Projects

As you can see, it has many options like RStudio Project, R Package, Quarto Project, Quarto Website, Quarto Blog and Quarto Book. This book is written using a Quarto Book project.

3.9 Installing Packages

You can install packages using the command `install.packages(package_name)`. For example, the command to install the `easystats` set of packages would be `install.packages(easystats)`.

3.10 Loading and Manipulating Data

In order to analyse the data, we need to load or import the data first. There are many possibilities as far as loading/importing data is concerned. If the data comes from a package, then you can directly

load the data using the `data(dataset_name)` command. For example, the command `data(mtcars)` will load the `mtcars` dataset into the environment. This dataset comes in-built in R.

Most of the data comes from external sources in different file formats like csv or Microsoft Excel. Data in csv file format can easily be read using the `read.csv(path_to_data_file)` command. There is `readr` package also, which is part of the `tidyverse` set of packages. This package `readr` can import csv data and load it in the environment as a `tibble`. The native data format in R is `data.frame`, and `tibble` is “advanced version of `data.frame` according to the developers of the `tidyverse` set of packages.

Similarly, to import data from Microsoft Excel file, there is `readxl` package, which is also a part of `tidyverse`. Just like `readr`, `readxl` would also import the data in form of a `tibble`. Both the packages also share similar interface to import data.

The packages `foreign` and `haven` are useful for importing data from other statistical packages like SPSS, SAS and STATA.

Another noteworthy package is `data.table`, which can import data from many formats. Its function `fread` is versatile and can read data in many formats.

3.10.1. Manipulating Data

Although R is capable of data manipulation without using any other packages, some of the packages created especially to facilitate data manipulation makes the process easy and understandable for others. In our examples, we will use `tidyverse` set of packages for data manipulation. In particular, two of its packages `dplyr` and `tidyr` make this process quite easy.

3.10.2. Selecting Columns from Dataset

We can use the `select` command from `dplyr` for selecting columns from a dataset.

```
library(tidyverse)

library(conflicted)

conflict_prefer("lag", "dplyr")

conflict_prefer("tidy", "broom")

conflict_prefer("augment", "broom")

mtcars |> select(am:carb) |> head() |> knitr::kable()
```

Table 3.2.: Selecting Columns from am to carb from mtcars dataset

	am	gear	carb
Mazda RX4	1	4	4
Mazda RX4 Wag	1	4	4
Datsun 710	1	4	1
Hornet 4 Drive	0	3	1
Hornet Sportabout	0	3	2
Valiant	0	3	1

In the above command, we first loaded the `tidyverse` set of packages, so that its functionality is available in the R session. Then we took the built-in dataset `mtcars`, and then selected the columns from `am` to `carb` in the dataset. The `|>` is called the `pipe operator` in R. It was introduced in R version 4.1. Before that, there was another pipe operator (`%>%`) in the `magrittr` package, which is also a part of `tidyverse`. To understand how pipe operator makes coding easy and readable,

imagine you want to take $g(f(x))$, which reads g of f of x . It means you want to take a value x , apply the function f on that value, and on the output, you want to apply another function g . Before the introduction of pipe operator, you would have written it as $g(f(x))$, but now after R 4.1, you can write it as `x |> f() |> g()`.

3.10.3. Selecting Rows from a Dataset (aka filtering)

We can use `filter` function from `dplyr` for filtering rows of a dataset. For example, we want to filter the rows of `mtcars` dataset, where `cyl` is equal to 6. We can do this by

```
mtcars |> dplyr::filter(cyl==6) |> head() |> knitr::kable()
```

Table 3.3.: Selecting Rows from mtcars dataset where value of cyl is 6

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Wag											
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4

We can see that in this code, we have used “=” twice. In R, “=” is an assignment operator. It means, when we write `cyl=6`, it assigns value 6 to the variable `cyl`. So, when we want to check equality, we have to use “==” instead of “=”.

3.10.4. Arrange Data

Using `dplyr`, it is easy to arrange or sort data in ascending or descending order of a column: use the `arrange` function for that.

```
mtcars |> arrange(mpg)|> select(mpg:disp) |> head() |> knitr::kable()
```

Table 3.4.: Arrange the Data in Ascending Order of `mpg`

	mpg	cyl	disp
Cadillac Fleetwood	10.4	8	472
Lincoln Continental	10.4	8	460
Camaro Z28	13.3	8	350
Duster 360	14.3	8	360
Chrysler Imperial	14.7	8	440
Maserati Bora	15.0	8	301

You can similarly arrange the data in descending order of `mpg` column in the `mtcars` dataset.

```
mtcars |> arrange(desc(mpg)) |> select(mpg:disp) |> head() |> knitr::kable()
```

Table 3.5.: Arrange the Data in Descending Order of `mpg`

	mpg	cyl	disp
Toyota Corolla	33.9	4	71.1
Fiat 128	32.4	4	78.7
Honda Civic	30.4	4	75.7

	mpg	cyl	disp
Lotus Europa	30.4	4	95.1
Fiat X1-9	27.3	4	79.0
Porsche 914-2	26.0	4	120.3

3.10.5. Create a New Variable

You can create a new variable using the `mutate` function in `dplyr` package. For example, you want to create a new variable, called `FuelEff`, which is defined as reciprocal of `mpg`, then you can do it as follows:

```
mtcars |> mutate(FuelEff = 1/mpg) |> select(am:FuelEff) |> head() |> knitr::kable()
```

Table 3.6.: Create a New Variable in `mtcars` dataset

	am	gear	carb	FuelEff
Mazda RX4	1	4	4	0.0476190
Mazda RX4 Wag	1	4	4	0.0476190
Datsun 710	1	4	1	0.0438596
Hornet 4 Drive	0	3	1	0.0467290
Hornet Sportabout	0	3	2	0.0534759
Valiant	0	3	1	0.0552486

3.11 Summarize a Data

Suppose you want to see what is the average `mpg` in the whole dataset? Remember `mpg` stands for miles per gallon. So, this column shows the fuel efficiency of various cars. So, you can use `summary` function from `dplyr` to get this answer.

```
mtcars |> summarise("Average MPG" = mean(mpg)) |> knitr::kable()
```

Table 3.7.: Average Miles Per Gallon of Cars in USA

Average MPG
20.09062

However, this function `summarise` becomes more powerful when combined with another function `group_by`. For example, you want to know if there is difference in mean `mpg` for different level of `cyl`. You can see it using the following code:

```
mtcars |> group_by(cyl) |> summarise("Average MPG" = mean(mpg)) |> knitr::kable()
```

Table 3.8.: Miles Per Gallon for Different Values of Cylinder

cyl	Average MPG
4	26.66364
6	19.74286
8	15.10000

The `group_by` argument changes the unit of analysis. For example, if we write the code `mtcars`

`|> summarize(mean_mpg = mean(mpg))`, then the unit of analysis is all the the observations of the `mpg` column. But when we introduce `group_by`, by writing `mtcars |> group_by(cyl) |> summarise("Average MPG" = mean(mpg))`, then the unit of analysis is each level of `cyl` variable. Now the average of `mpg` is calculated for each level of `cyl`, instead of calculating it for the whole lot of observation.

4

Simple Linear Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Understanding the principles of simple linear regression*
 - *Performing simple linear regression in R for econometric analysis*
 - *Interpreting regression results in the context of economic variables*
 - *Assessing model assumptions and addressing violations*
 - *Practical examples and exercises using R*
-

```
library(easystats)

library(ggplot2)
```

4.1 Introduction

Linear regression is among the fundamental concepts in Econometrics. In linear regression, we try to estimate how much one variable will change, with response to a change in another variable. The controlled variable is called predictor or independent variable. The variable, which changes as a response to a change in the controlled variable is called response or dependent variable. In R, we denote such relationship with $y \sim x$, where y is the response and x is the predictor. The notation $y \sim x$ is read as “ y explained by x ”.

In mathematical terms, we are fitting a linear equation between x and y . When we write $y \sim x$, it means $y = ax + b$, and we need to find a and b from the data points given.

4.2 Unveiling Relationships: The Purpose of Linear Regression

Linear regression, a cornerstone of statistical analysis, empowers us to delve into the fascinating world of relationships between variables. Imagine yourself as a data detective, piecing together clues to understand how one element influences another. This is precisely the role linear regression plays.

Suppose you’re an enterprising researcher curious about the link between study hours and exam scores. Linear regression offers a method to uncover this connection. By harnessing the power of this technique, you can not only **predict** potential exam scores based on the number of hours a student studies, but also gain valuable insights into the **strength** of this relationship.

Consider the following scenario: you establish a model depicting the relationship between study hours and exam scores. This model, visualized as a straight line, allows you to predict the anticipated exam score for a student who dedicates a specific number of hours to studying. This predictive power is undeniably valuable.

However, linear regression goes beyond mere prediction. It unveils the **underlying nature** of the relationship between variables. The slope of the line you establish serves as a vital clue. A steeper slope suggests that additional study hours translate into a more significant increase in exam scores. Conversely, a gentler slope indicates a less pronounced impact of study hours on exam performance.

In essence, linear regression equips us with a two-pronged approach:

1. **Prediction:** We can leverage the model to forecast outcomes based on new data points.
2. **Understanding Relationships:** By analyzing the slope and other properties of the line, we gain insights into how changes in one variable (study hours) influence the other (exam scores).

It's important to remember that the world isn't always perfectly linear. However, linear regression serves as a robust foundation for data analysis, providing a framework to explore connections and make informed decisions based on the information gleaned from our data.

Let's understand what it means for us in terms of our workflow. The workflow for the linear regression problem would be:

1. We would be given some observations of both - independent variable(s) and dependent variable.
2. we graph these data points, using a coordinate system (like Cartesian system). Each value is represented by a dot. Such a diagram is called a scatter-plot diagram.

3. After graphing the points onto a scatter plot diagram, linear regression analysis seeks to find the best-fit line to fit the points as closely as possible.
4. This best-fit line is a line, which minimizes the distance between the points falling above or below the lines.

[Linear Regression 101: What Is It And How Is It Done? | Fiverr](#)

4.3 Unveiling the Assumptions Behind Linear Regression

While linear regression offers a powerful lens for analyzing relationships, its effectiveness hinges on certain underlying assumptions. These assumptions, akin to the ground rules of the game, ensure the validity of the results we obtain. Let's delve into these key assumptions:

1. **Linearity:** The core principle of linear regression lies in its name - **linearity**. This assumption dictates that the relationship between the two variables we're investigating is **straight-line** in nature. In simpler terms, when one variable increases or decreases, the other variable changes in a consistent, proportional manner, much like points scattered along a straight line.
2. **Independence of Errors:** This assumption emphasizes that the **errors** (the difference between the actual data point and the value predicted by the model) are **independent** of each other. Imagine each error term as a random fluctuation; these fluctuations shouldn't be influenced by the errors associated with other data points.
3. **Homoscedasticity:** This tongue-twister of a term refers to the notion of **constant variance** of the errors. In simpler terms, the spread of the errors (how far they deviate from the predicted value) should be **consistent** across all levels of the independent variable (the variable we believe causes the change in the other variable).

4. **Normality of Errors:** This assumption posits that the errors associated with our data follow a **normal distribution**, often referred to as a bell curve. This characteristic distribution helps ensure the reliability of the statistical tests we perform to assess the model's validity.

By adhering to these assumptions, we create a solid foundation for interpreting the results of our linear regression analysis. However, it's crucial to remember that real-world data may not always perfectly satisfy these assumptions. Fortunately, there are techniques to address potential violations and ensure the robustness of our analysis. We'll explore these strategies in the upcoming chapters!

4.4 Example in R

For understanding regression, we can generate a dummy dataset, and create a regression model on the basis of the data.

```
dummy <- data.frame(y = rnorm(100), x = rpois(100, 3))
```

In the code above, we have created a dummy data. The `rnorm(n)` function generates n number of data points based on normal distribution. The default mean and standard deviation are 0 and 1 respectively. Similarly, the `rpois(n, 1)` function generates n number of data points based on poisson distribution, with `lambda = 1`. We can plot the data to explore it visually. We will use `ggplot2` package to create this scatterplot diagram.

```
ggplot(dummy) + aes(x, y) + geom_point() + labs(title = "Scatterplot Diagram of X and Y")
```

Now we can create a regression model from this dataset.

```
model1 <- lm(y ~ x, data = dummy)
```

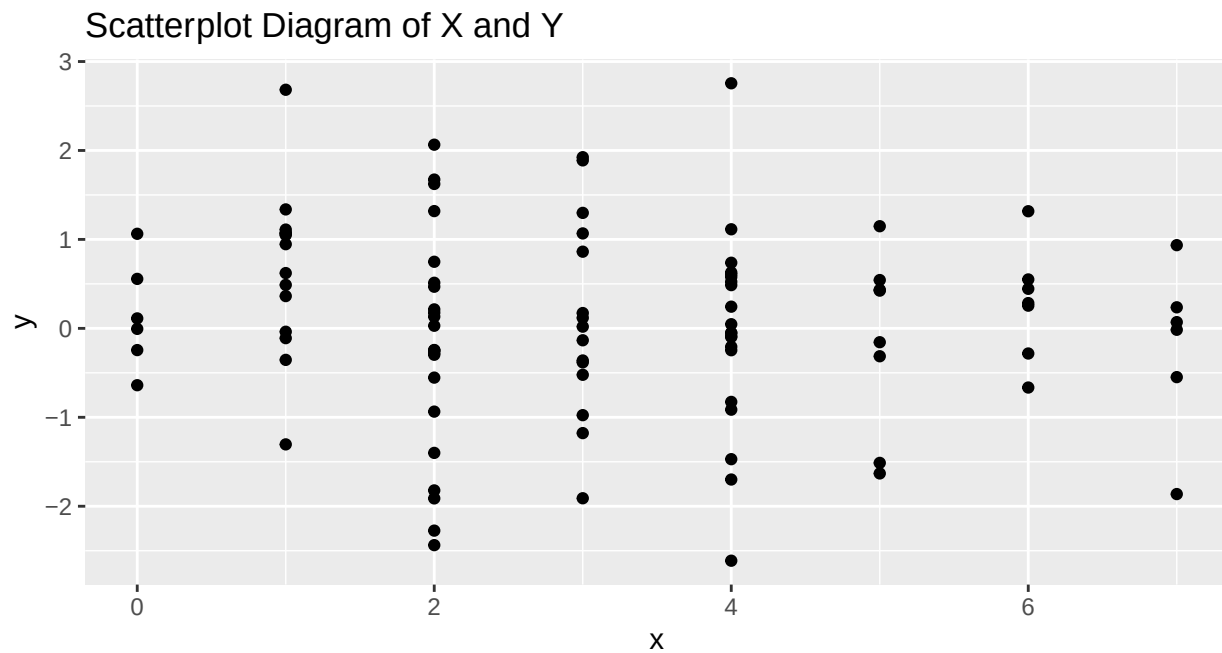


Figure 4.1.: Scatterplot of X and Y

This model stores a lot of information, but it is difficult to understand and get the meaning out of it. So, one way is to use the `summary` function on this `model1` object.

```
summary(model1)
```

Call:

```
lm(formula = y ~ x, data = dummy)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.6342	-0.4445	0.0324	0.6090	2.7336

Coefficients:

Estimate	Std. Error	t value	Pr(> t)
----------	------------	---------	----------

```
(Intercept)  0.24693    0.20632    1.197    0.234
x            -0.05627    0.05672   -0.992    0.324
```

Residual standard error: 1.051 on 98 degrees of freedom

Multiple R-squared: 0.009944, Adjusted R-squared: -0.0001589

F-statistic: 0.9843 on 1 and 98 DF, p-value: 0.3236

The problem with this summary object is that it is not a data.frame. It is difficult to put this object in a research paper or any other academic submission. So, we can use the `easystats` packages for this. The `model_parameters` function would show us the parameters (aka coefficients) of the model, and the `model_performance` function will show us the effectiveness of the regression model. We can also use the `display` function to beautify the table in the output.

```
model11 |> model_parameters() |>
  display(format = "markdown", caption = "Regression Parameters")
```

Parameter	Coefficient	SE	95% CI	t(98)	p
(Intercept)	0.25	0.21	(-0.16, 0.66)	1.20	0.234
x	-0.06	0.06	(-0.17, 0.06)	-0.99	0.324

```
model11 |> model_performance() |>
  display(format = "markdown", caption = "Regression Effectiveness")
```

Table 4.3.: Regression Effectiveness

Table 4.3.: Regression Effectiveness						
AIC	AICc	BIC	R2	R2 (adj.)	RMSE	Sigma
297.71	297.96	305.53	9.94e-03	-1.59e-04	1.04	1.05

4.4.1. Interpretation of Regression Model

- The intercept is statistically non-significant and negative (beta = -0.16, 95% CI [-0.59, 0.27], t(98) = -0.74, p = 0.461; Std. beta = -5.50e-18, 95% CI [-0.20, 0.20]).

- The effect of x is statistically non-significant and positive (beta = 0.07, 95% CI [-0.04, 0.17], t(98) = 1.24, p = 0.219; Std. beta = 0.12, 95% CI [-0.07, 0.32]).

The model explains a statistically not significant and very

weak proportion of variance ($R^2 = 0.02$, $F(1, 98) = 1.53$, p =

0.219, adj. $R^2 = 5.32e-03$).

4.4.2. Non-Linear Relationship in Regression

Sometimes, the relationship between the dependent and independent variable is not linear, but quadratic. This can be understood either from the scatter plot diagram or from theoretical considerations. In mathematical terms, the relationship is

$$Y = \beta_0 + \beta_1 * X^2 + \gamma \quad (4.1)$$

where X and Y are independent and dependent variables, respectively. This is still considered “Linear Regression”, because we are interested in the linearity of the coefficient term (i.e., β_0 and β_1), not the linearity of the variables. In the equation above, both the coefficients (β_0 and β_1) are linear, and so is the error term, γ in this equation.

4.5 Interval Estimation and Hypothesis Testing

Linear regression empowers us to understand relationships and make predictions. But remember, the world of data isn’t perfect, and our models are based on samples, not the entire population. This inherent uncertainty is where **interval estimation** steps in.

Imagine you estimated a straight line to represent the connection between house size (square footage) and selling price. This line predicts the average selling price for a given house size. But what if a single house is much larger or smaller than those in your sample? The predicted price from the model might not be the exact selling price.

Interval estimation tackles this by providing a **range of plausible values** for the outcome variable (selling price) at a specific level of the independent variable (house size). This range, called a **confidence interval**, gives us a sense of the **precision** of our prediction.

Here’s how it works:

1. We choose a **confidence level**, typically 90% or 95%. This indicates the probability that the true value falls within the constructed interval.
2. We leverage the concept of the **standard error of the estimate**. This statistical measure reflects the average amount our predictions might deviate from the actual values.

3. By incorporating the chosen confidence level and the standard error, we calculate the **upper and lower bounds** of the confidence interval. This range represents the values within which we are confident (based on the chosen level) the true outcome variable will lie for a given independent variable value.

What does this tell us?

A narrower confidence interval indicates a more precise prediction. The model is more certain about the range of potential selling prices for a specific house size. Conversely, a wider interval reflects greater uncertainty. The model can't pinpoint the exact selling price as confidently.

Interval estimation is crucial because:

- It acknowledges the inherent uncertainty in our predictions.
- It allows us to communicate the **precision** of our findings.
- It helps us assess the **reliability** of the model for predictions.

By incorporating interval estimation, we paint a more nuanced picture when interpreting the results of linear regression analysis. We not only provide a predicted value but also quantify the associated uncertainty, making our analysis more informative and reliable.

Let us understand interval estimation with an example of a real dataset. We will use the `mtcars` dataset, which comes bundled with R.

```
mod1 <- lm(mpg~disp, data = mtcars)
```

In the code above, we fitted a model on the `mtcars` dataset, where Miles Per Gallon, or `mpg` is explained by Displacement (cubic inches), or `disp` variable. The coefficient for `disp` is -0.0412151. The 95% confidence interval of the coefficients can be found using the `confint` function in R.

Table 4.4.: Confidence Interval of Model Coefficients

	2.5 %	97.5 %
(Intercept)	27.0884	32.1113
disp	-0.0508	-0.0316

```
confint(mod1) |> kbl(digits = 4, booktabs = TRUE, align = "c")
```

In the code above, the `confint` function is used to find out the confidence interval, while the other functions are used to format the resulting table. The last row of the table suggests that the 95% confidence interval for the `disp` variable is between (-0.0508) and (-0.0316).

4.6 Hypothesis Testing

Linear regression equips us to explore relationships and make predictions, but are these connections merely due to chance, or do they hold true significance? This is where **hypothesis testing** enters the scene. Imagine you suspect a link between coffee consumption and alertness levels. Regression analysis can tell you if there's a relationship, but hypothesis testing helps determine if this connection is statistically **meaningful**.

Here's the framework:

Formulating Hypotheses:

Null Hypothesis (H₀): This hypothesis proposes that there's **no significant relationship** between the independent and dependent variables (e.g., coffee consumption has no impact on alertness levels).

Alternative Hypothesis (H_a): This hypothesis stands in opposition to the null hypothesis, suggesting that a **significant relationship** exists (e.g., coffee consumption does influence alertness

levels).

Choosing a Significance Level: This threshold, typically set at 5%, determines how likely we are to reject the null hypothesis even when it's actually true (a false positive). A lower significance level indicates a stricter test, requiring stronger evidence to reject the null hypothesis.

Test Statistic and p-value: We employ statistical tests like the t-test or F-test to calculate a **test statistic**. This value reflects the strength of the evidence against the null hypothesis. A higher (absolute) value suggests a stronger case for rejecting the null hypothesis.

We then calculate the **p-value**, which represents the probability of observing a test statistic as extreme or more extreme than the one we calculated, assuming the null hypothesis is true. A lower p-value indicates less likelihood of observing such an extreme result by chance alone.

Decision Making: Based on the chosen significance level and the p-value, we make a decision:

Reject H_0 : If the p-value is lower than the significance level, we reject the null hypothesis. There's sufficient evidence to suggest a significant relationship between the variables.

Fail to Reject H_0 : If the p-value is greater than the significance level, we fail to reject the null hypothesis. We cannot conclude a statistically significant relationship at this level of significance. However, this doesn't necessarily imply there's no connection; it just means we don't have enough evidence to disprove the null hypothesis.

R automatically performs a hypothesis testing, along with presenting regression summary. For example, in the Table 4.2, the last column titled "p" conducts a two-way hypothesis testing for the t-value shown in the previous column, titled "t(98)", (where 98 number shows the degrees of freedom) whether the t-value is significantly different from zero. In this case, the null and the alternative hypothesis are as follows:

$$H_0 : t = 0$$

$$H_1 : t \neq 0$$

Given that both the p-values (0.234 and 0.324) are greater than the significance level (0.05), we fail to reject the null hypothesis. In other words, we believe that both, the intercept and x, are statistically significant in order to explain the variation in the dependent variable, y.

4.7 Prediction

Linear regression not only unveils relationships between variables but also empowers us to **predict** future outcomes. Imagine you're a business analyst trying to forecast sales based on advertising spending. R, a powerful statistical programming language, provides tools to leverage your regression model for predictions.

Here's a breakdown of the prediction process with an example using R code:

1. Sample Data and Model Fitting:

```
# Simulate some data for advertising spend (ad_spend) and sales (sales)

set.seed(123)

ad_spend <- rnorm(100, mean = 100, sd = 15)

sales <- 50 + 2 * ad_spend + rnorm(100, mean = 0, sd = 10)

# Fit the linear regression model

model <- lm(sales ~ ad_spend)
```

In this code:

- We simulate data for advertising spend and sales.
- The `lm` function fits a linear regression model where `sales` is the dependent variable and `ad_spend` is the independent variable.

2. Predicting Sales for a New Advertising Spend Value:

```
# Predict sales for an ad spend of $120

new_spend <- 120

predicted_sales <- predict(model, newdata = data.frame(ad_spend = new_spend))

cat("Predicted sales for an ad spend of $", new_spend, ":", predicted_sales, sep = "")
```

Predicted sales for an ad spend of \$120:288.2723

This code:

- Defines a new advertising spend value of \$120.
- The `predict` function leverages the fitted model (`model`) to predict sales for this new data point (represented as a data frame with the new `ad_spend` value).
- The `cat` function displays the predicted sales value.

This indicates that based on the model, sales are predicted to be around \$290 for an advertising spend of \$120.

The key difference between prediction intervals and confidence intervals in the `predict` function of R lies in what they are trying to estimate:

- **Prediction Interval:** Focuses on a **single future observation**. It estimates the range within which a **new individual value** of the dependent variable is likely to fall, given a specific value of the independent variable.
- **Confidence Interval:** Focuses on the **mean prediction**. It estimates the range that's likely to contain the **average value** of the dependent variable for a specific value of the independent variable.

Here's a breakdown to illustrate the concept:

Scenario: Predicting sales based on advertising spend.

- **Confidence Interval:** If you set a 95% confidence interval for predicted sales at an ad spend of \$120, it indicates that you're 95% confident the **average sales** for that advertising spend will fall within that interval.
- **Prediction Interval:** A 95% prediction interval for the same scenario estimates the range that's likely to encompass a **single new sales value** at an ad spend of \$120.

R Code Demonstration:

```
# Simulate data for advertising spend (ad_spend) and sales (sales)

set.seed(123)

ad_spend <- rnorm(100, mean = 100, sd = 15)

sales <- 50 + 2 * ad_spend + rnorm(100, mean = 0, sd = 10)

# Fit the linear regression model

model <- lm(sales ~ ad_spend)

# New advertising spend value
```

```
new_spend <- 120

# Confidence Interval for Mean Prediction

conf_interval <- predict(model, newdata = data.frame(ad_spend = new_spend), interval = "conf")

# Prediction Interval for a New Observation

prediction_interval <- predict(model, newdata = data.frame(ad_spend = new_spend), interval = "prediction")

cat("Predicted sales for an ad spend of $", new_spend, ":", summary(predicted_sales), sep = "\n")
```

Predicted sales for an ad spend of \$

120

:

288.2723

288.2723

288.2723

288.2723

288.2723

288.2723

```
cat("95% Confidence Interval for Mean Prediction:", conf_interval[,c(1,3)], sep = "\n")
```

95% Confidence Interval for Mean Prediction:

288.2723

291.5374

```
cat("95% Prediction Interval for a New Observation:", prediction_interval[,c(1,3)], sep = "\n")
```

95% Prediction Interval for a New Observation:

288.2723

307.8106

Key Points:

- The confidence interval for the mean prediction is much narrower than the prediction interval for a new observation. This reflects the inherent uncertainty in predicting a single future value compared to the average sales across multiple observations.
- Prediction intervals are generally wider because they account for the variability around the average sales figure, along with the uncertainty in the model itself.

In conclusion, choosing between a confidence interval and a prediction interval depends on your goal:

- If you're interested in the **average sales** for a specific advertising spend, use the confidence interval.
- If you want to estimate the range for a **single new sales value**, then the prediction interval is more appropriate.

5

Multiple Linear Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Extending regression analysis to multiple independent variables*
 - *Building and interpreting multiple linear regression models in R*
 - *Handling multicollinearity and selecting significant predictors in an economic context*
 - *Model evaluation and diagnostics in econometric regression*
 - *Application of multiple linear regression in economic analysis using R*
-

5.0.1. Introduction

In the previous chapter, we saw simple linear regression, in which we attempted to understand the relationship between two variables - one called independent variable and the other called the dependent variable. Multiple linear regression is a statistical technique that allows us to analyze the relationship between two or more independent variables and a dependent variable. It is an extension of simple linear regression, which only involves one independent variable. Multiple linear regression is used to predict the value of a dependent variable based on the values of two or more independent variables. It is commonly used in fields such as economics, finance, and social sciences.

5.1 Assumptions of Multiple Linear Regression

There are a few assumptions of multiple linear regression. They are as follows:

1. Linearity of coefficients and error terms

The coefficients (β) and the error terms (ϵ) are linear. The variables themselves might be linear or polynomials. But the coefficients and error terms must be linear.

2. The error term (ϵ) has population mean zero.

In particular, the error term is normally distributed with the population mean of zero. If the population mean of the error term is other than zero, it means at least some part of the error term is predictable. Such a predictable part should be kept in the regression equation itself, not in error term. The error term should contain only random error, which can be attributed to chance. When the population mean is not equal to zero, statisticians call it “bias”.

3. None of the independent variables is correlated with the error term.

If any of the independent variable is correlated with the error term, it means it is possible to predict the error term. So, in such case, it is better to put this information in the regression equation itself. As we said in the previous point, the error term should contain only the random error, which explains the variability of the dependent variable. If the error term also explains the variability of any of the independent variable, it is not an error per se.

4. Observations in the error term are independent of one another (i.e., not correlated with one another). If error terms are correlated, it means it is possible to predict the error term of the next observation, using the information of the error term in the first observation. And any information, which can be used to predict the error term, should be put in the regression equation itself. The error terms should contain only the random component.
5. The variance in the error terms remain constant for all the observations. This assumption is also called homoskedasticity. PENDING
6. There is no perfect correlation between any of the independent variable with the dependent variable. If there is a perfect correlation between any two variables, then one of the two variables is unnecessary.

5.2 Multiple Linear Regression: A Practical Example

Let's look at the dataset from the book "Basic Econometrics" by Damodar Gujarati, et al (The Special Indian Economy Edition). The Table 7.6 shows consumption of Chicken in 22 states and union territories in India. The variables are as follows:

- Y: Consumption of chicken (in kg)
- X_2 : Price of chicken (Rs. Per kg)

- X_3 : Price of fish (Rs. Per kg)
- X_4 : Per capita income (Rs. Thousands at 1993-94 prices)

(Note: You need to download the `Gujarati5sie` package from github using the command `devtools::install_github("bhattmaulik/Gujarati5sie")`. Once you have installed this package, you can check the documentation of any of the table contained in the package using the `?table` syntax. For example, `?Table7.6` will show you the documentation for the Table 7.6 in the package.)

We can load the dataset, and inspect the variables in the dataset.

```
#devtools::install_github("bhattmaulik/Gujarati5sie")

#to download the "Gujarati5sie" package

#which contains the dataset

#install.packages("GGally")

#to install the GGally package

# library(easystats)

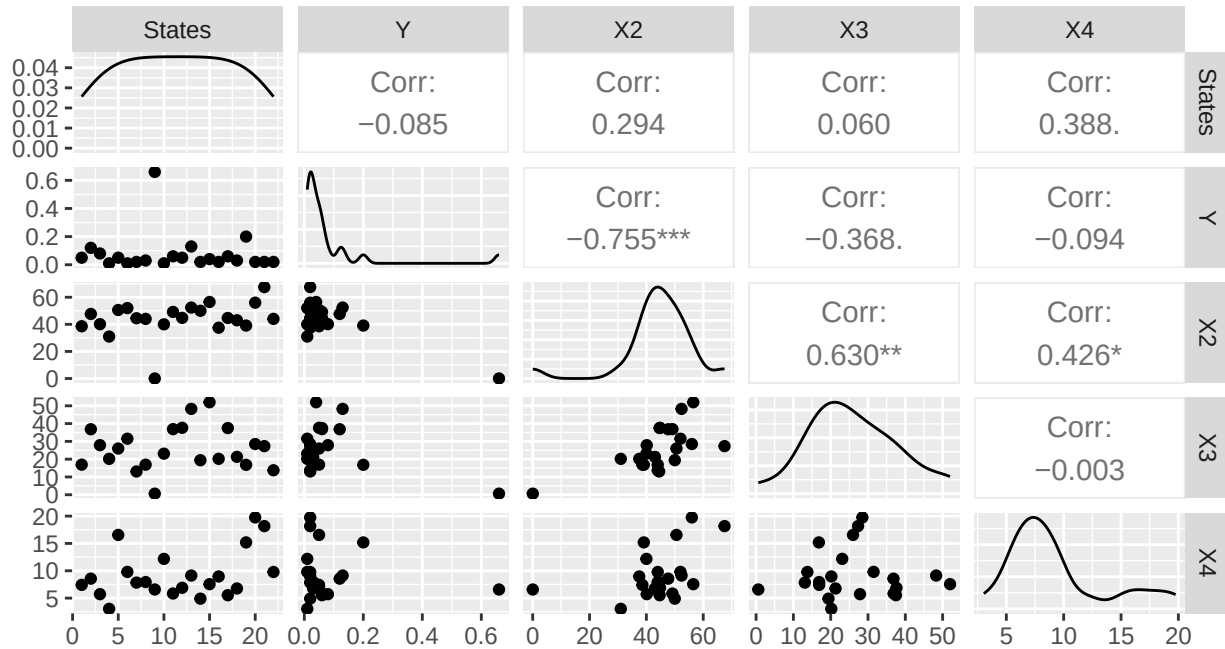
# library(Gujarati5sie)

# library(GGally)

data("Table7.6")

#to load the dataset from Table 7.6

ggpairs(Table7.6)
```



```
#to check correlation among variables in the dataset
```

We can see that Y has negative correlation with X2 and X3, and slightly negative correlation with X4 also. This looks **usual more or less**, because according to theory, the consumption of a commodity would have negative relationship with its price. Fish is a competitor for chicken, so consumption of chicken should have positive correlation with the price of fish, which is not the case in this dataset.

Let's fit a linear regression model on the data.

```
mod1 <- lm(Y~X2+X3+X4, data = Table7.6)
```

We can see that multiple regression also requires the `lm` function, just like classical linear regression model. To add more variables in the regression model, we have to use the `+` sign on the right hand side of the `~` sign.

We can see the model parameters and model diagnostics using the `easystats` set of packages.

```
model_parameters(mod1) |>
  display(format = "markdown", caption = "Multiple Regression Parameters")
```

Table 5.2.: Multiple Regression Parameters

Parameter	Coefficient	SE	95% CI	t(18)	p
(Intercept)	0.41	0.07	(0.28, 0.55)	6.27	< .001
X2	-0.01	2.18e-03	(-0.02, -8.04e-03)	-5.78	< .001
X3	4.11e-03	2.06e-03	(-2.17e-04, 8.44e-03)	2.00	0.061
X4	0.01	4.82e-03	(2.29e-03, 0.02)	2.58	0.019

PENDING

Model Building Strategies

Often, regression model building needs multiple models. In the first step, the analyst needs to know the possibly relevant independent variables, which might affect the dependent variable. Then the data is collected for these variables. After data collection, the analyst would fit a regression model, and find coefficients for the model. But that's not the end of the regression. The analyst would need to check whether the model fulfills the assumptions of the regression model. If there are violations, the analyst would need to create another model. Often, a step-wise regression process is adopted.

In a step-wise regression model, independent variables would be added or removed one-by-one and the analyst would monitor its impact on the decision criteria (often, R^2 or adjusted R^2 , but it could be AIC or BIC also). The analyst would choose the model, which optimizes the decision criteria: for R^2 and adjusted R^2 , the more the better, while for AIC and BIC, the less is the better.

also see pdf on steps to follow in multiple regression model building

[13 Multiple Regression and Model Building | Quantitative Research Methods for Political Science, Public Policy and Public Administration: 4th Edition With Applications in R \(bookdown.org\)](#)

Model Diagnostics

[Chapter 13 Multiple Regression Models | Introduction to Statistics and Data Science \(nps.edu\)](#)

[Diagnostics_for_multiple_regression \(stanford.edu\)](#)

Interpretation of Results of Multiple Linear Regression

[Linear Regression in R | A Step-by-Step Guide & Examples \(scribbr.com\)](#)

[Reading a Regression Table: A Guide for Students | Steven V. Miller \(svmiller.com\)](#)

Multicollinearity and Variable Selection

[Multicollinearity in Regression Analysis: Problems, Detection, and Solutions - Statistics By Jim](#)

[Multicollinearity in Regression Analysis: Everything You Need to Know | upGrad blog](#)

Model Selection Strategies

[Model Selection – Test Science 3.0](#)

[Evaluation metrics & Model Selection in Linear Regression | by NVS Yashwanth | Towards Data Science](#)

6

Regression Analysis with Dummy Variables

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Incorporating categorical variables in regression analysis*
 - *Creating and interpreting dummy variables in R*
 - *Dummy variable pitfalls and remedies in econometric modeling*
 - *Examples and case studies of dummy variable regression in economics using R*
-

- Incorporating categorical variables in regression analysis

- Creating and interpreting dummy variables in R
- Dummy variable pitfalls and remedies in econometric modeling
- Examples and case studies of dummy variable regression in economics using R

[Regression with Categorical Variables: Dummy Coding Essentials in R - Articles - STHDA](#)

[Microsoft Word - Use and Interpretation of Dummy Variables.doc \(rhul.ac.uk\)](#)

6.1 Dummy Variables

Categorical variables are the variables, which divide the observations in different groups. These categorical variables are also called factors, dummy variables, or qualitative variables. For example, gender of respondent (Male v/s Female) is a factor or dummy variable. The two values “Male” and “Female” are the levels of the factor. There are packages in R, e.g., **forcats**, which deal with factor variables.

6.2 Dummy Variables in Regression

In a linear regression model, the dependent variable would be a numeric (or continuous) variable. But on the independent variable side, one or more variable(s) can be factors. In such cases, we need to take some extra steps. These steps typically involve “coding” dummy variables. We can understand this by a practical example in R.

The **penguins** dataset from the **palmerpenguins** package contains several factor variables, like **species**, **island** and **sex**. We will use **species** as the independent variable, and the **body_mass_g** (which captures the body mass in grams) as the dependent variables. By using the **levels** function, we can see that there are three levels of the **species** variable: Adelie, Chinstrap, and Gentoo.

```
library(palmerpenguins)

penguins <- palmerpenguins::penguins

levels(penguins$species)
```

```
[1] "Adelie"      "Chinstrap" "Gentoo"
```

```
mod1 <- lm(body_mass_g ~ species, data = penguins)

summary(mod1)
```

Call:

```
lm(formula = body_mass_g ~ species, data = penguins)
```

Residuals:

Min	1Q	Median	3Q	Max
-1126.02	-333.09	-33.09	316.91	1223.98

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	3700.66	37.62	98.37	<2e-16 ***
speciesChinstrap	32.43	67.51	0.48	0.631
speciesGentoo	1375.35	56.15	24.50	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 462.3 on 339 degrees of freedom

(2 observations deleted due to missingness)

Multiple R-squared: 0.6697, Adjusted R-squared: 0.6677

F-statistic: 343.6 on 2 and 339 DF, p-value: < 2.2e-16

In the code above, we created a linear regression model, where `body_mass_g` is the dependent variable, and the `species` is the independent variable. When we take `summary` of this regression model, we can see that there are three coefficients: Intercept, and the two levels of `species` variable, Chinstrap and Gentoo. The third level of the `species` variable (i.e., Adelie) is missing in this equation.

This is because Adelie is the base level in the equation. You can understand this from the `contrasts` function in the code given below:

```
contrasts(penguins$species)
```

	Chinstrap	Gentoo
Adelie	0	0
Chinstrap	1	0
Gentoo	0	1

You can see that Adelie is coded as 0 (zero) in this table. Therefore, the interpretation of the coefficients would also change accordingly. The coefficient of 32.43 for `speciesChinstrap` means that on average, the body mass of penguins belonging to Chinstrap species would be 32.43 grams more than a penguin belonging to the Adelie species (assuming all other factors equal). Similarly, the body mass of a penguin belonging to Gentoo category would be more than the Adelie category of penguin by 1,375.35 grams on average, assuming other factors equal.

6.3 Recoding Dummy Variables

A natural question arises here: R had, by default, assigned Adelie category as the base level in this dataset. But is it possible to change which category would be considered the base level? In other words, the Adelie category was assigned the code of 0 in this table. Can we manually decide which category is coded 0?

It is possible to do this. This process is called re-coding the factor levels. We can use the `fct_relevel` function from the `forcats` package for this. (Of course, there is `relevel` function if base R also. But as we are using `tidyverse` and `easystats` packages in this book throughout, it is better to use `forcats` package, which shares the core philosophy of `tidyverse`, and works well with other `tidyverse` packages.) We will have to load the `forcats` package first, because it is not loaded by default when `tidyverse` is loaded.

```
library(forcats)#load the package

penguins1 <- penguins |> mutate(species = fct_relevel(species, "Gentoo"))

mod2 <- lm(body_mass_g~species, data = penguins1)

summary(mod2)
```

Call:

```
lm(formula = body_mass_g ~ species, data = penguins1)
```

Residuals:

Min	1Q	Median	3Q	Max
-1126.02	-333.09	-33.09	316.91	1223.98

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5076.02	41.68	121.78	<2e-16 ***
speciesAdelie	-1375.35	56.15	-24.50	<2e-16 ***
speciesChinstrap	-1342.93	69.86	-19.22	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 462.3 on 339 degrees of freedom

(2 observations deleted due to missingness)

Multiple R-squared: 0.6697, Adjusted R-squared: 0.6677

F-statistic: 343.6 on 2 and 339 DF, p-value: < 2.2e-16

We can see that the coefficients in this new model have changed. Now, there is no **Gentoo** in the model, but there are **Adelie** and **Chinstrap**. The numbers have changed too. The coefficient for **speciesAdelie** is -1,375.35, which means that on average, the body mass of **Adelie** category of penguin is less than that of **Gentoo** penguin by 1,375.35 grams, assuming other factors are held constant.

6.4 Dummy Variable Trap

In the previous example, we saw that there are three levels in the **species** variable, but only two of them enter into the regression model. In other words, in the regression summary, we can see only two of them. This is to avoid the situation called “dummy variable trap”.

Let us understand this situation with an example. We will create three dummy variables in this example, and use these three dummy variables as independent variables in a regression model.

```
penguins2 <- penguins |>
  dplyr::mutate(specie_adelie = if_else(species == "Adelie",1,0),specie_chinstrap = if_else(
mod3 <- lm(body_mass_g~specie_adelie + specie_chinstrap + specie_gentoo, data = penguins2)
summary(mod3)
```

Call:

```
lm(formula = body_mass_g ~ specie_adelie + specie_chinstrap +
    specie_gentoo, data = penguins2)
```

Residuals:

Min	1Q	Median	3Q	Max
-1126.02	-333.09	-33.09	316.91	1223.98

Coefficients: (1 not defined because of singularities)

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5076.02	41.68	121.78	<2e-16 ***
specie_adelie	-1375.35	56.15	-24.50	<2e-16 ***
specie_chinstrap	-1342.93	69.86	-19.22	<2e-16 ***
specie_gentoo	NA	NA	NA	NA

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 462.3 on 339 degrees of freedom

(2 observations deleted due to missingness)

Multiple R-squared: 0.6697, Adjusted R-squared: 0.6677

F-statistic: 343.6 on 2 and 339 DF, p-value: < 2.2e-16

You can see in the summary that the coefficient for `specie_gentoo` variable is NA. That means this regression model is not useful for any prediction or inference.

This is because one essential condition for linear regression is that there is no multicollinearity in the data. That means, if one variable can be expressed as a linear combination of two or more variables, the regression model involving these variables will throw error, just like our newly created model did.

In the code below, I have selected 6 random rows from the `penguins2` dataset that we just created, and converted it into matrix form. The first column in the matrix is the coefficient of the constant term (called `intercept` in the regression summary). The other three columns in the matrix are the three columns that we created: `specie_adelie`, `specie_chinstrap` and `specie_gentoo`.

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

In this matrix, we can see that the sum of the last three columns is always 1, which is the coefficient of the intercept term. Thus, the first column can be expressed as the sum of the last three columns. Thus, the matrix doesn't have full rank. Therefore, no solution can be found for this equation. That's the reason we are seeing **NA** in the regression summary when we include all the three dummy variables. There is a simple solution to avoid this dummy variable trap: exclude one dummy variable out of the three we created.

7

Heteroscedasticity and Robust Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Understanding heteroscedasticity and its implications*
 - *Addressing heteroscedasticity using robust regression techniques in R*
 - *Interpreting robust regression results in an economic context*
 - *Practical examples and exercises showcasing robust regression in econometrics*
-

- Understanding heteroscedasticity and its implications

- Addressing heteroscedasticity using robust regression techniques in R
- Interpreting robust regression results in an economic context
- Practical examples and exercises showcasing robust regression in econometrics

Heteroscedasticity

In the Chapter 4, we noted that one of the conditions for linear regression is to have equal variance at all the levels of the variables. In other words, the actual values of the observations, and the values of the observation predicted by the regression equation should be close enough throughout the regression line.

Imagine you're studying the relationship between apartment size (square footage) and rent price. Linear regression helps you understand this connection, but what if the spread of errors (differences between predicted and actual rent) isn't consistent across all apartment sizes? This inconsistency is precisely what heteroscedasticity refers to.

Let's break it down:

- **Homoscedasticity (the good kind):** In a perfect world, the errors in your regression analysis would be **homoscedastic**. This means the spread of the errors (how far they deviate from the predicted value) would be **constant** across all values of the independent variable (apartment size in our example). Imagine a nice, even spread of dots around the regression line.
- **Heteroscedasticity (the not-so-good kind):** In reality, data often exhibits heteroscedasticity. The spread of the errors might be **unequal** across different values of the independent variable. This can manifest as a funnel-shaped or fan-shaped pattern of errors around the

regression line. For example, errors for smaller apartments might be tightly clustered, while errors for larger apartments might be more scattered.

Why is heteroscedasticity a problem?

Our linear regression assumptions rely on homoscedasticity. When errors are unequal, it can lead to:

- **Inaccurate standard errors:** The estimated standard errors of the coefficients (measures of how precise our estimates are) might not be reliable. This makes it difficult to assess the significance of the relationships between variables.
- **Underestimation or overestimation of p-values:** P-values help us determine if the observed relationships are due to chance. Heteroscedasticity can lead to unreliable p-values, potentially masking true relationships or suggesting significance where there isn't any.

Here's an R example to illustrate:

```
set.seed(123)

# Simulate data with homoscedasticity
x <- rnorm(100, mean = 50, sd = 5)
y <- 2 * x + rnorm(100, mean = 0, sd = 3)

# Simulate data with heteroscedasticity
x2 <- rnorm(100, mean = 50, sd = 10)
y2 <- 2 * x2 + rnorm(100, mean = 0, sd = x2/2)

# Error term proportional to x2 in the above code

#create dataframe from the simulated data
hom_dataframe <- data.frame(x = x, y = y)
```

```

het_dataframe <- data.frame(x = x2, y = y2)

#create models

model_homo <- lm(y ~ x, data = hom_dataframe)

model_het <- lm(y2 ~ x2, data = het_dataframe)

#use "augment" function from broom package to find the actual and predicted values

hom_augment <- broom::augment(model_homo)

het_augment <- broom::augment(model_het)

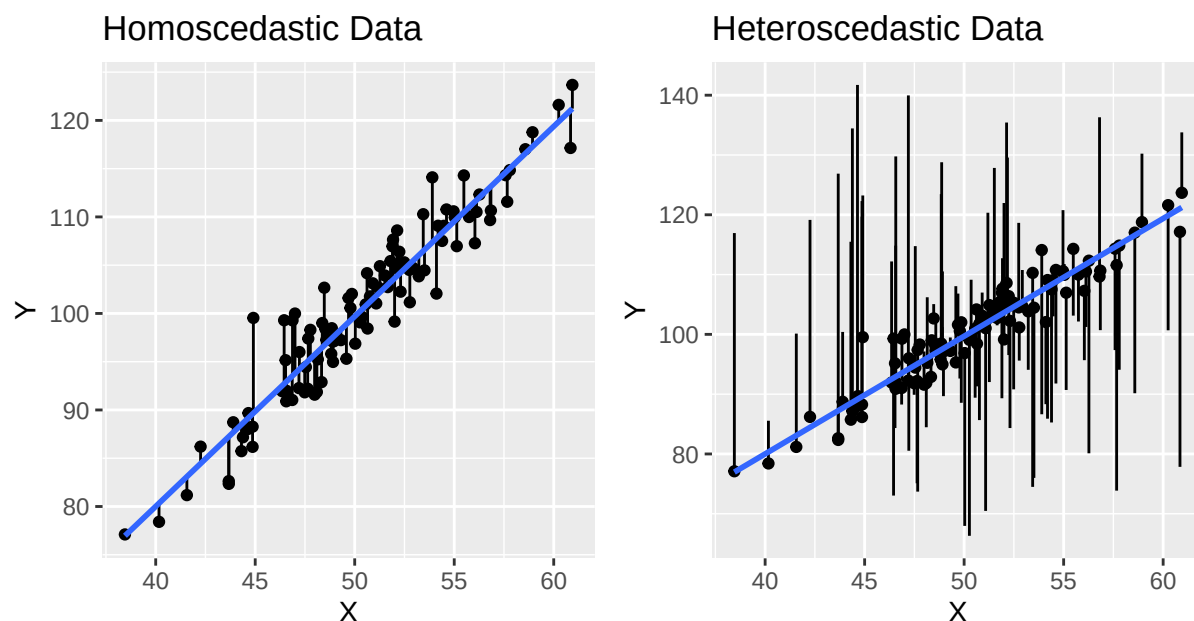
# Visualize

plot_hom <- ggplot(hom_augment, aes(x,y))+geom_point()+geom_linerange(aes(ymin = y,ymax = .f

plot_het <- ggplot(het_augment, aes(x,y))+geom_point()+geom_linerange(aes(ymin = y,ymax = .f

plot_hom+plot_het

```



Run this code and compare the plots. In the first plot (homoscedastic), the errors should be scattered relatively evenly around the regression line. In the second plot (heteroscedastic), the spread of errors should be wider for larger values of x_2 .

7.1 Formal Test for Heteroscedasticity

In the section above, we used visual method to determine whether heteroscedasticity exists in a model or not. We also have formal tests to determine if that's the case. The Breusch-Pagan test formally checks whether heteroscedasticity exists in a linear regression model or not. The `bptest` function from the `lmtest` package implements this test in R. In this test, we need to check the p-value. If the p-value is less than 0.05, we can confirm that heteroscedasticity exists in the model.

```
bptest(model_homo)
```

```
studentized Breusch-Pagan test
```

```
data:  model_homo
```

```
BP = 1.2665, df = 1, p-value = 0.2604
```

```
bptest(model_het)
```

```
studentized Breusch-Pagan test
```

```
data:  model_het
```

BP = 8.9835, df = 1, p-value = 0.002724

In the code above, we can see that the p-value for the `bptest` for the first model is much larger than 0.05, which means that there is no heteroscedasticity in this model. But in the second case, the p-value is 0.002, which suggests that there is heteroscedasticity for sure.

PENDING

Ways to deal with heteroscedasticity

8

Time Series Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Introduction to time series data in econometrics*
 - *Time series regression models in R for economic analysis*
 - *Dealing with autocorrelation and lagged variables*
 - *Forecasting with time series regression models in R*
 - *Applications of time series regression in economic forecasting*
-

- Introduction to time series data in econometrics

- Time series regression models in R for economic analysis
- Dealing with autocorrelation and lagged variables
- Forecasting with time series regression models in R
- Applications of time series regression in economic forecasting

9

Introduction to Logistic Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Basics of logistic regression in econometrics*
 - *Estimating logistic regression models in R*
 - *Interpreting logistic regression coefficients and odds ratios*
 - *Applications of logistic regression in economic research using R*
-

- Basics of logistic regression in econometrics

- Estimating logistic regression models in R
- Interpreting logistic regression coefficients and odds ratios
- Applications of logistic regression in economic research using R

10

Model Evaluation and Selection

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Evaluating model performance and goodness-of-fit measures in econometrics*
 - *Validation techniques for econometric regression models*
 - *Comparing and selecting models using information criteria*
 - *Cross-validation and bootstrapping for robust model assessment in econometrics*
-
- Evaluating model performance and goodness-of-fit measures in econometrics
 - Validation techniques for econometric regression models
 - Comparing and selecting models using information criteria

- Cross-validation and bootstrapping for robust model assessment in econometrics

11

Practical Tips and Resources for Econometric Regression

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Data preparation and preprocessing tips for econometric analysis*
 - *Handling missing data and outliers in regression analysis*
 - *Dealing with endogeneity and instrumental variables*
 - *Additional resources for further learning and practice in econometrics with R*
-

- Data preparation and preprocessing tips for econometric analysis

- Handling missing data and outliers in regression analysis
- Dealing with endogeneity and instrumental variables
- Additional resources for further learning and practice in econometrics with R

It is said that a data scientist spends 70% of his/her time in data cleaning rather than data analysis. Therefore, in this chapter, we will discuss various techniques to prepare and preprocess the data for econometric analysis. In the chapter Chapter 3, we introduced some methods of data pre-processing, which were essentially sub-setting the data from a larger set and creating new variables from the existing ones. In this chapter, we will deal with data cleaning, finding missing values, finding columns where a date is stored as a number (for example, importing data from Microsoft Excel), etc.

11.1 Data Cleaning

The very first step of data cleaning starts with the data import itself. When you import data from `csv` file (using `readr::read_csv` function) or Microsoft Excel file (using the `readxl::read_excel` function), you have the option of cleaning the names directly while importing the data. Both of these functions have an argument called `.name_repair`, which determines how the column names would be changed after importing the data. The default method is the keep the names `unique`, which means, if the dataset contains multiple columns having the same name, it would change the name to `name1`, `name2`, etc. It is also possible to provide a function to this argument. We can see an example in the code below.

Table 11.2.: Making Unique Names in a Dataset

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa

Table 11.3.: Making Clean Names in a Dataset using janitor package

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa

```
library(readxl)#load the package

read_excel(readxl_example("datasets.xlsx"), .name_repair = "unique") |> head() |>
  kbl()
```

Now, instead of keeping the default `unique` names, we can change it using `make_clean_names` function from the `janitor` package. For this to work, you must have installed the `janitor` package in your system.

```
read_excel(readxl_example("datasets.xlsx"),
  .name_repair = janitor::make_clean_names) |> head() |>
  kbl()
```

This function (`make_clean_names`) converts all the names to lower case, and changes to underscore instead of dots in the column names. It also makes some more changes like writing “percent” instead of the percentage symbol (%) in the column name (if any column name has it), etc.

11.2 Finding Missing Values

Often, the data will have blank or missing values. It is easy to find out if your data has missing values, using the `summary` function.

```
dummy_data <- data.frame(A = c(1:5), B = c(6:8, NA, NA))  
  
summary(dummy_data)
```

	A	B
Min.	:1	Min. :6.0
1st Qu.:	2	1st Qu.:6.5
Median	:3	Median :7.0
Mean	:3	Mean :7.0
3rd Qu.:	4	3rd Qu.:7.5
Max.	:5	Max. :8.0
		NA's :2

In the code above, we created a dummy data, in which column B contains two NA values. When we take `summary(dummy_data)`, it will show that there are two NAs in the data. Thus, if your data has NAs, the `summary` will show it easily. Therefore, it is a good practice to look at the summary of the data as soon as you import it.

Summary is a powerful function in R. It will change the output depending on the input you write in the brackets. Even if the input is a dataset, it will change the output depending upon the types of the data in the columns of the data. In the previous example, we saw that the output showed items like minimum, maximum, median, mean and NA values. If we had only text values in a column, we could see different output.

Table 11.4.: Summary of 'Species' column of 'iris' dataset

	summary.iris.Species.
setosa	50
versicolor	50
virginica	50

```
data.frame(summary(iris$Species)) |> kbl(  
  
)
```

In the code above, we took summary of the `Species` column of the `iris` dataset, which comes in-built in R. This column has only text data in it. So, it shows the frequency of each values (setosa, versicolor and virginica).

11.3 Date Stored as Numbers

Microsoft Excel is a popular spreadsheet program. So, you might have to import data from Microsoft Excel file format quite often. However, Microsoft Excel stores date as numbers internally. Therefore, when you open the Microsoft Excel file, on the screen you might see a column as date, but when you import the data in other program (like R), it would be imported as a serial number. In such cases, the `excel_numeric_to_date` function from the `janitor` package comes handy.

```
janitor::excel_numeric_to_date(45450)
```

```
[1] "2024-06-07"
```

The code above converts the numeric value 45450 to a date format. By default, Microsoft Excel follows 1900 date system, which means 1st January, 1900 is set as the base date, with the value of 0. All other dates are difference of that date from the base date. The date 7-June-2024 is

45450 days away from the base date, therefore the value of this date is 45450. The function `excel_numeric_to_date` identifies this and converts this numeric value to Date format.

11.4 Identifying Outliers in Data

12

Conclusion

Status

This chapter is currently a dumping ground for ideas, and we don't recommend reading it.

Learning Objectives:

In this chapter, you will learn to

- *Summary of the key concepts covered in the book*
 - *Importance of regression analysis in econometrics and economic research*
 - *Encouragement for further exploration and application of econometric regression using R*
-
- Summary of the key concepts covered in the book
 - Importance of regression analysis in econometrics and economic research
 - Encouragement for further exploration and application of econometric regression using R



R packages for econometric regression analysis and additional resources

This book could never be completed without using many packages. The most notable of the packages include tidyverse, easystats, AER, lmtest, fpp3, gujarati5sie, etc. These packages can be installed using the following commands in R.

```
RA_packages <- c("AER",  
                 "easystats",  
                 "fpp3",  
                 "lmtest",  
                 "ggthemes",
```

```
      "palmerpenguins",  
      "patchwork",  
      "here","fs",  
      "knitr",  
      "kableExtra")  
  
install.packages(RA_packages)
```

The commands above install the packages into your R system. However, the functionality of these packages are not added into your R session just because you installed these packages. You have to load the required packages specifically whenever you need them.

Imagine you are building a home. You have completed electrification in your home. But just because you have a fan or an AC in your home doesn't mean the fan or AC would start automatically. You have to switch on the appliance whenever you need it. Similarly, you will have to load the R packages into your session whenever you need those packages. You can load the package using the `library(package_name)` command. For example, the command to load **easystats** set of packages would be

```
library(easystats)
```

When you run the `install.packages(package_name)` command, R installs the package from Comprehensive R Archive Network (CRAN), which is the highest authority to decide about R. However, there are many packages, which are not available on CRAN. These packages can be downloaded from code sharing platform Github.

To install packages from Github, you need to install one of the three packages first: **remotes**, **devtools** or **pak**. After that, you can easily install packages from Github also. Apart from such

packages, the development versions of regular packages (which are available on CRAN) can also be downloaded from Github. For example, if you want to install the package “gujarati5sie”, which is a package containing data from the 5th edition of the book “Basic Econometrics” written by Damodar Gujarati and others, then you can do it as follows:

```
#Install devtools, remotes or pak

install.packages("remotes")#or install.packages("devtools")

#or install.packages("pak")

#Load the package

library(remotes)#or library(devtools)

#or library(pak)

#download the gujarati5sie package

#using one of the above packages

remotes::install_github("bhattmaulik/Gujarati5sie")

#or devtools::install_github("bhattmaulik/Gujarati5sie")

#or pak::pak("bhattmaulik/Gujarati5sie")
```




Data sets used in the book's examples

B.0.1. Introduction

In this textbook, we have relied upon datasets available in various packages. The base R also comes with some in-built datasets like `mtcars` and `iris`. We have used these datasets in some of the examples. We have also used `penguins` dataset from the `palmerpenguins` package. In the Appendix [A](#), you will find how to install these packages.

After installing the package, you will also need to load the package in the system every time you want to use it. This can be done using the `library` command as shown in the Appendix [A](#). After the package is loaded, you can use the datasets contained in the package.



How to Build This Book Locally

C.0.1. Introduction

If you want to build this book locally on your computer, please download the entire code from Github. The first step is to visit the website www.github.com/bhattmaulik/RegressionAnalysis. Here, you will find the option to download the entire book in a zip file, as shown below.

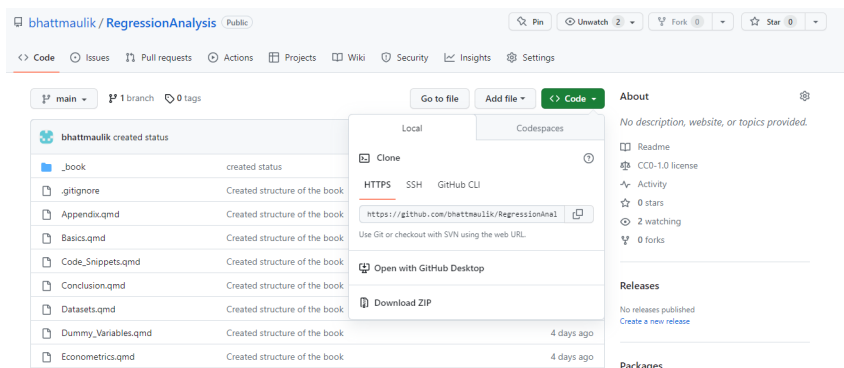


Figure C.1.: Download the book from Github

After you download the zip file, unzip it. This will create a folder in your computer. Within this folder, double click on the “RegressionAnalysis.Rproj” file. This will open the whole project in

RStudio.

After opening this project, go to “Build” pane in RStudio. This pane is generally on the top right of RStudio along with Environment, History, Connections, Git and Tutorial. In the build pane, click on “Render Book” and select “HTML format”.

You can also choose to build the book in PDF format, but for that you will need additional software called `tinytex`. In order to build a book in PDF format, R also needs `LaTeX` compiler. There are two popular `LaTeX` compilers: `TeX Live` and `MikTeX`. However, they have their own set of problems for R users. The `LaTeX` compiler `tinytex` attempts to solve many of them.

If you want to build the PDF book, you can install `tinytex` through terminal in RStudio using the command `quarto install tinytex`.

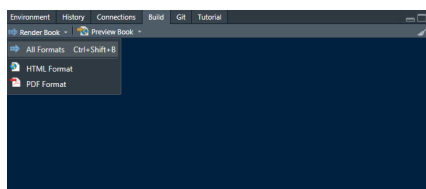


Figure C.2.: Build book using `tinytex`

If you can’t install it this way, or if you want to install it through traditional way using R code, you can use the following code:

```
install.packages("tinytex")

library(tinytex)

tinytex::install_tinytex()

#if you want to install all the LaTeX packages,

#you can modify the command to
```

```
#tinytex::install_tinytex(bundle = "TinyTeX-2")
```

This confuses some readers because there are two `tinytex` in this code: the first `tinytex` is the `tinytex` R package. The other `tinytex` is the LaTeX compiler `tinytex`. So, when we write `library(tinytex)`, we are calling the R package `tinytex`. And when we use the command `install_tinytex`, we are installing the LaTeX compiler `tinytex` using the R package `tinytex`. The benefit of this approach is that you get to select which bundle you want to install. By default, you get to install the bundle `TinyTeX-1`, which contains only the most necessary LaTeX packages. But if you choose the bundle `TinyTeX-2`, you can download all the LaTeX packages.

References

- Arnold, Jeffrey B. 2021. “Ggthemes: Extra Themes, Scales and Geoms for Ggplot2.” Manual. <https://CRAN.R-project.org/package=ggthemes>.
- Gujarati, Damodar. 2017. *Basic Econometrics*. 5th ed. India: McGraw Hill Education.
- Hester, Jim, Hadley Wickham, and Gábor Csárdi. 2023. “Fs: Cross-Platform File System Operations Based on Libuv.” Manual. <https://CRAN.R-project.org/package=fs>.
- Hyndman, Rob. 2023. “Fpp3: Data for ”Forecasting: Principles and Practice” (3rd Edition).” Manual. <https://CRAN.R-project.org/package=fpp3>.
- Kleiber, Christian, and Achim Zeileis. 2008. *Applied Econometrics with r*. New York: Springer-Verlag. <https://CRAN.R-project.org/package=AER>.
- Lüdecke, Daniel, Mattan S. Ben-Shachar, Indrajeet Patil, Brenton M. Wiernik, Etienne Bacher, Rémi Thériault, and Dominique Makowski. 2022. “Easystats: Framework for Easy Statistical Modeling, Visualization, and Reporting.” *CRAN*. <https://easystats.github.io/easystats/>.
- Müller, Kirill. 2020. “Here: A Simpler Way to Find Your Files.” Manual. <https://CRAN.R-project.org/package=here>.
- Pedersen, Thomas Lin. 2022. “Patchwork: The Composer of Plots.” Manual. <https://CRAN.R-project.org/package=patchwork>.

- Xie, Yihui. 2014. “Knitr: A Comprehensive Tool for Reproducible Research in r.” In *Implementing Reproducible Computational Research*, edited by Victoria Stodden, Friedrich Leisch, and Roger D. Peng. Chapman; Hall/CRC.
- . 2015. *Dynamic Documents with r and Knitr*. 2nd ed. Boca Raton, Florida: Chapman; Hall/CRC. <https://yihui.org/knitr/>.
- . 2023. “Knitr: A General-Purpose Package for Dynamic Report Generation in r.” Manual. <https://yihui.org/knitr/>.
- Zeileis, Achim, and Torsten Hothorn. 2002. “Diagnostic Checking in Regression Relationships.” *R News* 2 (3): 7–10. <https://CRAN.R-project.org/doc/Rnews/>.
- Zhu, Hao. 2021. “kableExtra: Construct Complex Table with Kable and Pipe Syntax.” Manual. <https://CRAN.R-project.org/package=kableExtra>.